Operational semantics for Prolog with Cut in Rocq and its application to determinacy analysis

Anonymous Author(s)

Abstract

We mechanize two operational semantics for Prolog with cut and prove their equivalence in the Rocq prover.

The first semantics is the one introduced by Debray and Mishra [8] for studying the optimizations employed by Prolog abstract machines [24, 30]. This semantics is based on a stack of alternatives, where the cut operator is represented by a pointer to a shorter stack.

The second semantics is novel and based on And-Or trees, in which the structure of alternative computations is made explicit and preserved throughout execution. This enables an explicit characterization of the cut operator as a function that prunes selected branches of the tree. The tree-based semantics allows us to formulate introduction and elimination rules for disjunction in the presence of cut, which, as expected, differ from those of classical logic.

As a case study, we use the tree-based semantics to prove the soundness of the determinacy analysis introduced in [1]. By exploiting the equivalence between the two semantics, we relate this result to the stack-based semantics used in efficient implementations of logic programming such as [10].

Keywords: Semantics, Prolog, Elpi, Determinacy Analysis, Rocq, Coq

1 Introduction

Backtracking is a compelling feature of logic programming, but it is also a double-edged sword. On the one hand, it makes it possible to write concise code that searches for a solution or enumerates all solutions. On the other hand, backtracking is often a source of poor performance and can make logic programs harder to debug. To mitigate these problems, the logic programming community has studied various forms of determinacy analysis [1, 9, 14, 15, 17, 18, 25] that identify logic programs that are deterministic, i.e., that compute functions rather than relations: they commit to their first solution and leave no *choice points* (places where backtracking could resume). A choice point corresponds to a suspended *alternative* computation; in the following, we use *choice point* and *alternative* interchangeably. The control operator *cut* allows the programmer to discard choice points and is therefore pervasively used to write deterministic code, but it is also well known to make the semantics of the programming language depart from a purely logical reading, thereby making reasoning about code harder.

Intuitively, nondeterminism arises when a computation can be completed using two different rules. This situation is typically ruled out statically by performing two complementary checks. The first one, called *mutual exclusion*, checks that at most one rule can apply to a given query. This condition is satisfied if either (i) no two rules have overlapping heads, so that no query can unify with both, or (ii) the analysis accounts for the presence of cut in rule premises, so that once the cut is executed all remaining rules are discarded. The second check, called *rule-body checking*, verifies that each rule either (i) calls only functions, ensuring that no choice point remains after the premises of the chosen rule have been executed, or (ii) executes a cut after calling relations, in which case any choice points left by the call are discarded.

Most papers in the literature only provide paper proofs for the soundness of their analysis, with the exception of Kriener and King [17]. They mechanize in the Rocq prover (formerly the Coq proof assistant) a denotational semantics for Prolog with cut and use it to prove the soundness of an abstract-interpretation-based determinacy inference algorithm. In addition to their Rocq code being unavailable, this mechanization has two further shortcomings. First, the analysis does not scale to dynamic predicates (see [1, Sec. 8]), so it cannot be applied to λprolog [22, 23] which is our long term goal. Second, there is no mechanized link between their denotational semantics and the operational semantics of Debray and Mishra [8], which underlie standard Prolog abstract machines [2, 24, 30].

Contributions. In this paper we mechanize in Rocq two operational semantics for Prolog with cut. The first is the stack-based semantics given by Debray and Mishra. This semantics is closely related to actual Prolog implementations, but it makes reasoning about the scope of cut difficult: alternatives are stored in a stack, in chronological order, and the cut operator is represented as a pointer to a shorter stack. The relation between the rules being cut and the affected stack frames is therefore not immediately visible.

Conference'17, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

To address this limitation, we introduce a semantics based on And-Or trees, in which the history of the computation is represented in a structured way: given a goal, each applicable rule generates an Or-branch, which is in turn followed by an And-branch for each premise. This enables an explicit characterization of the cut operator as a function that prunes selected branches of the tree. The tree-based semantics allows us to formulate introduction and elimination rules for disjunction in the presence of cut, which, as expected, differ from those of classical logic. For example, $q_1 \vee q_2$ is not equivalent to $q_2 \vee q_1$, since q_2 may be cut away by q_1.

Finally, we use the tree-based semantics to prove the soundness of the determinacy analysis from [1]. In Rocq, we prove that any terminating execution of a predicate identified as deterministic introduces no additional choice points. Combined with the equivalence between the two semantics, this result implies that any call to a deterministic predicate leaves no choice points on the stack used by actual Prolog implementations, including the Elpi [10] λProlog interpreter, for which the static analysis of [1] was originally developed. Elpi serves as an extension language for the Rocq prover and has become an important component of the Rocq ecosystem [4–7, 11–13, 16, 27–29]; our result therefore further strengthens confidence in its implementation.

Our mechanization is axiom-free and is available at ANONYMIZED (included as supplementary material).

2 Preliminaries: syntax and informal semantics of cut

In the entire paper we use fig. 1 as our running example, and we start by describing how we represent a program like that one. In the concrete syntax, variables are written with capital letters, and predicate application follows the λ-calculus style (no parentheses).

The description of a program $\mathbb{P}$ is shared by both semantics and consists of a list of rules together with a signature environment. We delay the discussion of signatures to section 6 where we describe the static analysis that concerns them and we now focus on the rules.

A rule $\mathbb{R}$ is a pair $\mathbb{T} \times \text{list } \mathbb{A}$, where the first element, a term, represents the rule's head and the second, a list of atoms, represents the rule's premises.

$$\mathbb{T} := \text{Symb } \mathbb{N} \mid \text{Pred } \mathbb{N} \mid \text{Var } \mathbb{N} \mid \text{App } \mathbb{T}\ \mathbb{T}$$
$$\mathbb{A} := \text{cut} \mid \text{call } \mathbb{T}$$

An atom is either the cut operator or a call to a term. Terms include predicate symbols, data symbols, variables, and term application.

The syntax tree allows variables to be applied and predicates to be mixed with data. These higher-order features play no role in the current paper, which focuses on first-order logic programming, but they allow future extensions to full λProlog to be based on the same mechanization.

Variables, predicates and data symbols are identified by natural numbers. We represent substitutions (of type Σ) as finitely supported maps from variables to terms, using the finmap library [21]. The empty substitution is written ε, while the composition of two substitutions σ_1 and σ_2 with disjoint supports is written $\sigma_1 + \sigma_2$.

The backchaining operation, central to both semantics, applies all rules to a query term:

$$\text{backchain} : \mathbb{P} \to \mathcal{F}_v \to \mathbb{T} \to \Sigma \to (\mathcal{F}_v \times \text{list } (\Sigma \times \text{list } \mathbb{A}))$$

Since a rule may be applied multiple times, its variables must be freshened before each application. Accordingly, the backchain function takes a program, the set of variable names used so far ($\mathcal{F}_v$), a query term, and the current substitution. It returns an updated set of variable names together with the list of alternatives, i.e. the rules that apply. More precisely, for each applicable rule it returns the rule premises together with an updated substitution obtained by unifying the rule head with the query term. The unification function takes two terms t_1 and t_2, together with a substitution σ, as input, and optionally returns a substitution σ' that extends σ. If t_1 and t_2 unify starting from σ, then $\sigma'\ t_1 = \sigma'\ t_2$, where $\sigma\ t$ denotes substitution application and $=$ denotes syntactic equality.

To keep the mechanization axiom-free, we implement a unification algorithm, which we briefly describe in section 6 and prove sound and complete. Since the two semantics and the static analysis use the same unification algorithm, the equivalence proof is completely agnostic to the particular implementation chosen for that algorithm.

2.1 The cut operator

There are several variants of the cut operator in the literature. The most widely adopted one is the *hard cut* (see, e.g., the documentation of SWI-Prolog [26]). Whenever the backchain function returns a non-empty list of alternatives, the first is explored immediately, while the others are suspended as choice points. Within a given alternative, premises are processed from left to right, executing each atom in turn. When a cut is encountered, it discards (i) all choice points created by backchaining for the current call and (ii) all choice points created while executing the premises to the left of the cut.

For example, consider the program in fig. 1 and the query `g 0 R`. Both rules for the predicate `g` apply to this query. Backchaining therefore returns the following two alternatives:

$$[(\sigma_1, [\texttt{r A B, f B C, !}]),\ (\sigma_2, [\texttt{f D E, f E F}])]$$

with $\sigma_1 := \{\texttt{A} \mapsto \texttt{0}, \texttt{C} \mapsto \texttt{R}\}$ and $\sigma_2 := \{\texttt{D} \mapsto \texttt{0}, \texttt{F} \mapsto \texttt{R}\}$. For simplicity, we choose an example where variable refreshing plays no role, so we do not discuss the set $\mathcal{F}_v$ any further.

```
func f int -> int.     % f is a deterministic predicate.
f 1 2.                 % The two rules have non-verlapping
f 2 3.                 % heads, i.e. 1 != 2
```

```
pred r int -> int.     % r is a non-deterministic predicate.
r 0 0.                 % The heads overlap: the query for 0
r 0 1.                 % has three applicable rules.
r 0 2 :- !.
```

```
func g int -> int.        % g is a deterministic predicate.
g A C :- r A B, f B C, !. % If the first rule succeeds, the second rule and the choice points left by r are cut
g D F :- f D E, f E F.    % The second rule is the last one and only calls functions
```

Figure 1. Running example of a logic program in Elpi syntax

Execution continues with the first premise of the first alternative, namely `r A B` under σ_1, i.e. `r 0 B`. The remaining alternatives are stored as choice points. Backchaining `r 0 B` returns $[(\sigma_3, []), (\sigma_4, []), (\sigma_5, [!])]$ where $\sigma_3 := \sigma_1 + \{B \mapsto 0\}$, $\sigma_4 := \sigma_1 + \{B \mapsto 1\}$ and $\sigma_5 := \sigma_1 + \{B \mapsto 2\}$.

The next step of interpretation continues with `f B C` under σ_3, that is, `f 0 R`, and leaves $[(\sigma_4, []), (\sigma_5, [!])]$ as new choice points. This time, backchaining fails (no rule for `f` matches input 0). We therefore backtrack to the most recently introduced choice point and retry `f B C` under σ_4, i.e. `f 1 R`. This succeeds with $\sigma_6 := \sigma_4 + \{R \mapsto 2\}$.

We now reach the cut. At this point there are still two unexplored alternatives: one from the third rule for `r` and one from the second rule for g (respectively, σ_5 and σ_2). Both are discarded, because they were created when, or after, the first rule for g was selected. In contrast, a cut does not discard choice points created earlier; in this example, there are none. The final solution is: $\{A \mapsto 0, B \mapsto 1, C \mapsto R, R \mapsto 2\}$.

In sections 3 and 4, we provide fully mechanized operational semantics for Prolog with cut. They differ in how they represent the ongoing computation, in particular how alternatives are stored and how they are pruned by cut.

Notational conventions In the following section we note $\mathbb{B}$ for the boolean type. Following the Mathematical Components library [19], on which we depend, we use `x.1` and `x.2` to denote the first and second projection applied to the pair `x`.

3 Stack-based semantics

The stack-based semantics we present is essentially the one by Debray and Mishra in [8, Section 4.3]. The main difference is that we hide the search for applicable rules in the backchain auxiliary function.

The execution state is a stack of alternatives.

$$\text{alts} := \text{list } (\Sigma \times \text{goals}) \qquad \text{goals} := \text{list } (\mathbb{A} \times \text{alts})$$

Intuitively each stack item is a self-contained computation, coupling a substitution with all the goals to be solved. The datatypes of goals and alternatives are mutually recursive since a goal pairs an atom with a list of the *cut-to* alternatives. Still, these alternatives have a very different role: they have to be understood as pointers to lower levels of the stack itself. They are used to implement the cut, i.e. to prune the stack of alternatives.

The stack-based semantics is described by the run_s inductive predicate.

$$\text{run}_s : \mathbb{P} \to \mathcal{F}_v \to \text{alts} \to \text{option } (\Sigma \times \text{alts}) \to \texttt{Prop}$$

The run_s predicate relates a program, a set of variables and a list of alternatives to an optional result. None stands for failure, i.e., no solution, whereas Some (σ, a) represents success with σ being the solution and a as the list of not-yet-explored alternatives.

The rule system for run_s is given in fig. 2 and consists of four cases. We distinguish two main cases. If we run out of alternatives we fail: rule Fail_s stops the execution and returns None. If the list of alternatives is $(\sigma_0, h) :: a$, then we look at the list of goals h (under the substitution σ_0). If h is empty, rule Stop_s stops the execution with a success σ_0 leaving a as the unexplored alternatives. If h is not empty we look at its head goal (t, ca) where t is the atom and ca is its cut-to alternatives. If t is a cut, rule Cut_s continues to evaluate the tail of h with alternatives ca. Finally, when t is a call, rule Call_s backchains: for each rule it pairs each premise with the current stack of alternatives, and prepends the obtained goals to the tail of h. Note that if backchain returns the empty list, the second premise of rule Call_s amounts to exploring the next item in the current stack.

$$\mathcal{B}_S\ p\ v\ t\ \sigma\ a\ g = \text{ let } (v', rs) := \text{backchain } p\ v\ t\ \sigma \text{ in}$$
$$\left(v', \left[\sigma', ([(q, a) \mid q \in b] +\!\!+ g) \,\middle|\, (\sigma', b) \in rs\right]\right)$$

$$\frac{}{\text{run}_s\ p\ v\ ((\sigma,\ []) :: a)\ (\text{Some } (\sigma,\ a))}\ \text{Stop}_s$$

$$\frac{\text{run}_s\ p\ v\ ((\sigma,\ g) :: \text{ca})\ r}{\text{run}_s\ p\ v\ ((\sigma,\ (\text{cut},\ \text{ca}) :: g) :: \_)\ r}\ \text{Cut}_s$$

$$\frac{\mathcal{B}_S\ p\ v\ t\ \sigma\ a\ g = (v',\ \text{bs}) \qquad \text{run}_s\ p\ v'\ (\text{bs}\ +\!\!+\ a)\ r}{\text{run}_s\ p\ v\ ((\sigma,\ (\text{call } t,\ \_) :: g) :: a)\ r}\ \text{Call}_s$$

$$\frac{}{\text{run}_s\ \_\ \_\ []\ \text{None}}\ \text{Fail}_s$$

Figure 2. Rule system for run_s

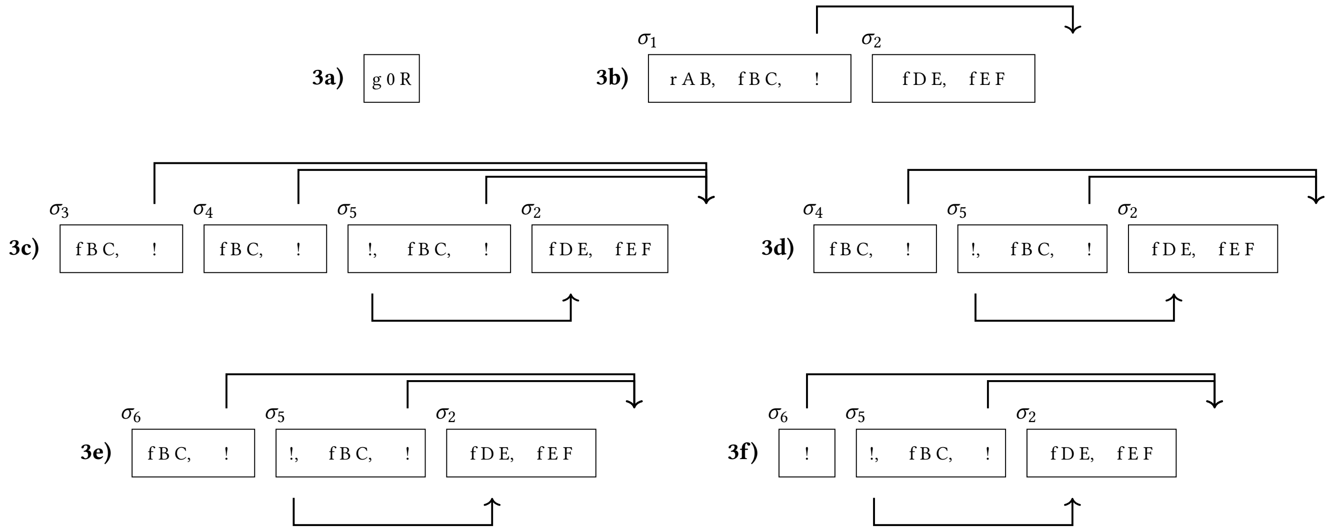


Figure 3. Example of execution

3.1 Example of execution

We propose an example of the execution of run_s in fig. 3. The arrows in the images represent the cut-to pointers.

We have not drawn arrows coming out of call atoms since the interpreter ignores the cut-to alternatives in this case. The evolution of the stack in fig. 3 mirrors the example in section 2.1: we evaluate the first goal of the first alternative. During backchaining, we add a pointer to the remaining alternatives to each new cut operator.

If a cut is evaluated, we replace the remaining alternatives with the cut-to alternatives. For example, in fig. 3f, evaluating the cut discards the second and third elements in the stack. Backtracking is performed by simply consuming the first alternative from the stack, as shown in fig. 3d.

4 And-Or Tree semantics

An And-Or tree is a stratified, variable-width tree, defined as follows:

$$\begin{aligned}\mathcal{T} := \;& \mathrm{Top} \mid \mathrm{Bot} \mid \mathrm{Unexplored}\ \mathbb{A}\\ & \mid \mathrm{Or}\ (\mathrm{option}\ \mathcal{T})\ \Sigma\ \mathcal{T} \mid \mathrm{And}\ \mathcal{T}\ (\mathrm{list}\ \mathbb{A})\ \mathcal{T}\end{aligned}$$

It consists of two kinds of layers that alternate: a disjunctive layer is followed by a conjunctive layer and vice versa. At the root (a query), the tree records a list of alternatives (an Or-layer); for each alternative, it records a list of subqueries to be solved (an And-layer). Intuitively, solving a query requires solving one alternative in the Or-layer, but all subqueries in the corresponding And-layer.

The tree is built incrementally. The Unexplored node represents an unexplored branch (an atom). When the atom is a call, we use the backchaining procedure from section 2 to compute the two layers to attach to the tree: alternatives form the Or-layer, and the premises of each applicable rule form the corresponding And-layer. This expansion step is performed by the step procedure described in section 4.2.

In addition to the Unexplored node we have two distinguished nodes, Bot and Top, written $\bot$ and $\top$, which represent the smallest failing and succeeding tree, respectively.

The Or node, written $A \lor B_\sigma$, represents the addition of an alternative B_σ to an existing disjunction A. The left subtree A is optional and is absent once that branch has been fully explored. The right alternative is paired with a substitution σ, which becomes the current substitution when backtracking to B.

The And node, written $A \land_{B_0} B$, represents the addition of a subquery B to the first choice point in A. We call B_0 the *reset point* for B: it represents the continuation for all alternatives of A except the first. That is, when backtracking occurs within A, the subtree B is replaced by the atoms in B_0. An example is shown in section 4.3.

4.1 The semantics

The tree is constructed incrementally by two recursive functions, step and prune. Both traverse the tree depth-first, left-to-right.

The type of these two functions is given below.

$$\begin{aligned}\mathrm{step} &: \mathbb{P} \to \mathcal{F}_v \to \Sigma \to \mathcal{T} \to (\mathcal{F}_v \times \mathrm{tag} \times \mathcal{T})\\ \mathrm{prune} &: \mathbb{B} \to \mathcal{T} \to \mathrm{option}\ \mathcal{T}\end{aligned}$$

The step routine, among its outputs, also returns a *tag*, whose definition is described in the very following section.

To avoid the constraint that all functions must terminate in Rocq, we define the transitive closure of step and prune as the inductive relation run_t. More precisely run_t iterates calls to step on incomplete trees to expand Unexplored nodes. When a tree fails it calls prune to find the next alternative and continues from there, if one exists. The node to be explored

```
Fixpoint next σ t : Σ * tree :=
  match t with
  | Unexplored _ | Bot | Top ⇒ (σ, t)
  | Or None σ′ B ⇒ next σ′ B
  | Or (Some A) _ _ ⇒ next σ A
  | And A _ B ⇒ let (σ′, pA) := next σ A in
                 if pA == Top then next σ′ B else (σ′, pA)
  end.
Definition next_subst σ t := fst (next σ t).
Definition next_tree t := snd (next ε t).
Definition success t := next_tree t == Top.
Definition failed t := next_tree t == Bot.
Definition incomplete t := match next_tree t with
  | Unexplored _ ⇒ true | _ ⇒ false end.
```

Figure 4. The next routine and derived functions

in a tree is retrieved thanks to next_tree (in fig. 4). When this node is Top we say the entire tree is a *success* (short ☑); when it is Bot we say the tree *failed* (short ⊠); otherwise the tree is *incomplete* (short ⊡).

A graphical representation of the execution of the running example on the tree-based semantics is shown in fig. 5. For compactness, we depict And and Or nodes as n-ary, with children associating to the right. The Bot node acts as the terminating element of an Or list, while Top is the terminating element of an And list. We mark with a black arrow the result of next_tree.

4.2 The step procedure

The step procedure takes as input a program, a set of variables, a substitution, and a tree, and returns an updated set of variables, a tag, and an updated tree.

The set of variables is assumed to contain all variables occurring in the tree. It is passed to backchain so that rules can be refreshed correctly.

The tag is implemented as follows:

tag := Expanded | CutBrothers | Failed | Success

It indicates which kind of step was performed. Failure and Success are returned for trees that satisfy, respectively, the failed and success tests. CutBrothers indicates the interpretation of a superficial cut: step was called on an And-layer and its next_tree is the cut atom. Finally, Expanded accounts for the interpretation of predicate calls and non-superficial cuts.

The step procedure leaves the tree unchanged when it is *success* or *failed*, while it expands Unexplored nodes by calling backchain, which returns a list l of alternatives. If l is empty, then the current call atom is replaced by Bot. Otherwise, it is replaced by a right-skewed tree of Or nodes, where each leaf corresponds to a list of atoms $e \in l$. If e is empty, the leaf is Top; otherwise, it is a right-skewed tree of And nodes.

Fig. 5 depicts the tree corresponding to the running example (section 2.1) as it undergoes calls to step and prune. We run query `q ∅ R` against the program P in fig. 1. Let c_0, c_1, and c_2 be the children of the root. By construction, c_0 is None, while c_1 and c_2 correspond respectively to the premises of rules r_1 and r_2 in P. Note that c_1 (resp. c_2) is associated with substitution σ_1 (resp. σ_2), whose values are the same as in section 2.1. Reset points are defined as follows: R_1 := [`f Y Z, !`], R_2 := [`!`], and R_3 := `f Y Z`. Intuitively, they record the lists of atoms used to generate the corresponding subtree. Other examples of step performing a tree expansion via backchain appear in figs. 5c to 5e.

4.2.1 The interpretation of a cut.

The step corresponding to a cut performs two actions: (i) the cut node is replaced by Top; and (ii) the subtrees in the scope of Cut are pruned. More precisely, (ii) prunes the left siblings of the Cut node (in the same And-layer, denoted $!_l$) and prunes the right uncles of Cut (in the one-level-up Or-layer, denoted $!_r$).

An example of the impact of cut is shown in fig. 5f. The red arrows indicate the scope of the cut and are labeled with $!_r$ and $!_l$ to indicate which pruning operation is applied to each pointed subtree. Note that the evaluation of a cut keeps the path leading to the cut node unchanged, in the sense that substitution σ_4 is preserved, while the path under substitution σ_5 (which also succeeds) is replaced by Bot. This amounts to discarding the third rule of predicate `r`.

4.3 The prune procedure

When backchain returns an empty list, step produces a tree that *failed* and the computation must backtrack. The prune procedure is responsible for producing the new tree from which the computation can continue.

In fig. 5d.1 we encounter a failure because no rule applies to the atom `f B C` under substitution σ_3, which maps `B` to `0`. The prune procedure prunes the path leading to this failure and resets the current sub-query to its original shape. More precisely, it kills the Top node under σ_3 (since that branch has been fully explored and produced no solution), and then replaces the Bot node with the information stored in the reset point R_1. This restores the call `f B C`, which can then be reconsidered under substitution σ_4.

Furthermore, prune serves another important purpose. Its behavior depends on the boolean flag it receives as input: prune false t recursively removes all failures (if any) from t, whereas prune true t removes the first success in t. For example, the tree in fig. 5f contains a successful path that maps `R` to `2`. Calling prune on this tree returns None, since there are no alternatives in the tree after the success.

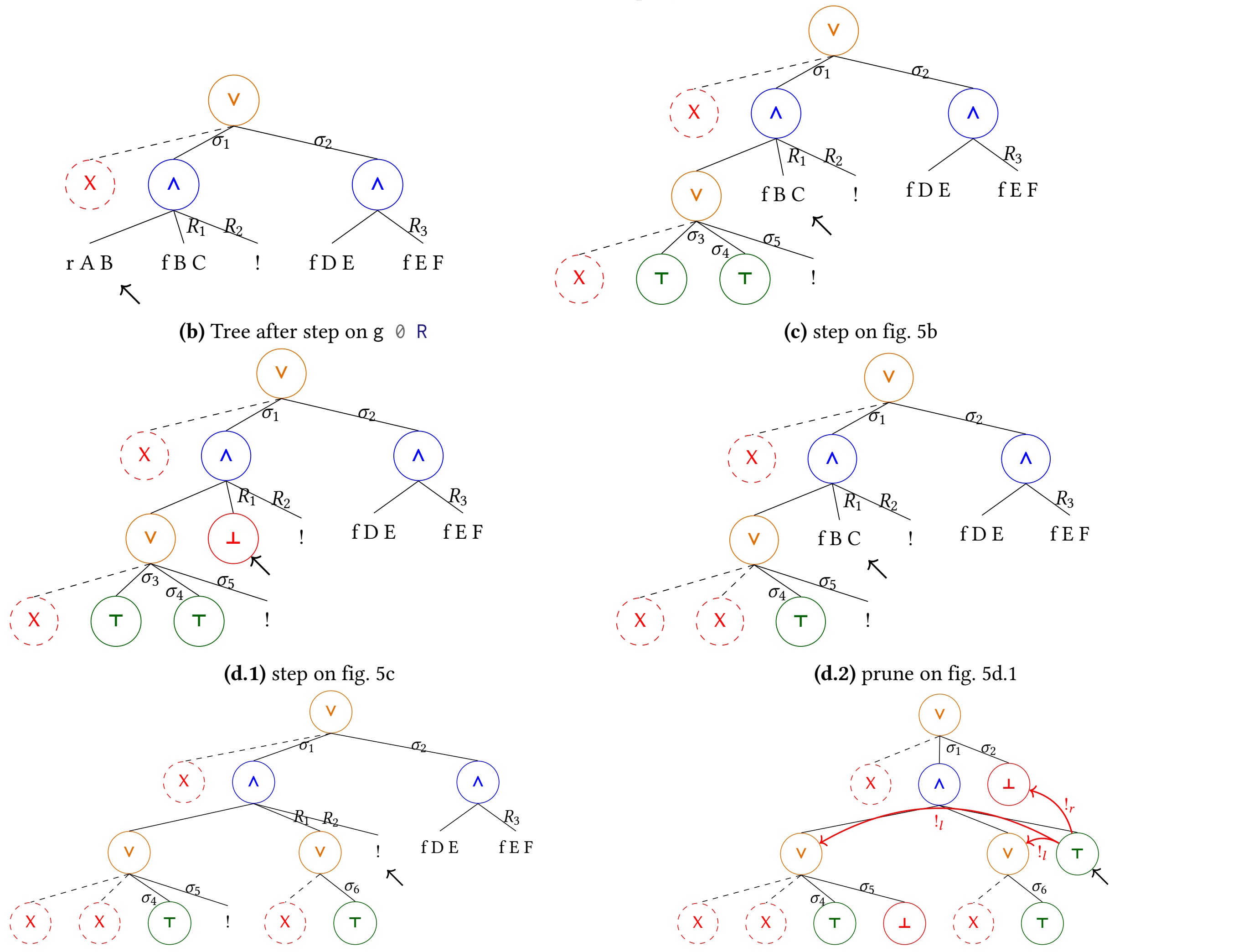
4.4 The run$_t$ relation and its properties

The inductive relation run$_t$ has the following type:

$$\mathrm{run_t} : \mathbb{P} \to \mathcal{F}_v \to \Sigma \to \mathbb{T} \to \mathrm{sol} \to \mathbb{B} \to \mathcal{F}_v \to \textbf{Prop}$$

g 0 R

(a) Initial query



(b) Tree after step on `g ∅ R`

(c) step on fig. 5b

(d.1) step on fig. 5c

(d.2) prune on fig. 5d.1

(e) step on fig. 5d.2

(f) step on fig. 5e

Figure 5. Execution in the tree semantics. The substitutions $\sigma_1 \dots \sigma_6$ are from section 2.1. R_1, R_2, and R_3 are reset points and correspond respectively to the lists of atoms [`f B C, !`], [`!`], and [`f E F`].

It associates a program, a set of variables, an initial substitution σ, and a tree t with a solution r, a boolean b, and an updated set of variable names v'.

The type of sol is defined as follows:

$$\text{sol} := \text{Zero} \mid \text{One } \Sigma \mid \text{Many } \Sigma\ \mathcal{T}$$

Zero represents a failing execution. One σ represents an interpretation producing exactly one solution, i.e. no backtracking is possible from the given state. It also carries the substitution σ that makes the interpretation succeed. Finally, Many σ t represents an execution in which the original state produces the solution σ, while t represents the remaining unexplored alternatives.

The boolean b records whether a superficial cut was evaluated during execution, and v' is the updated set of variables tracking freshly generated names.

The run_t predicate has five cases, depicted as inference rules in fig. 6a.

Rules $\text{Stop}_\text{t}^\text{o}$ and $\text{Stop}_\text{t}^\text{m}$ say that when the tree t is a success, the substitution σ' is extracted from t (starting from σ); depending on the result of prune true t – which cleans the tree of its successful path – we return One or Many, according to whether alternatives remain to be explored. Rule Step_t applies when next_tree t is an atom: step is invoked to evaluate t, and run_t is called recursively on the updated tree. When next_tree t is a failure, rule Back_t calls prune to clear

$$\frac{\boxed{\checkmark}\ \mathrm{t} \quad \text{next\_subst}\ \sigma\ t \rightsquigarrow \sigma' \quad \text{prune true}\ t \rightsquigarrow \text{None}}{\text{run}_\text{t}\ p\ v\ \sigma\ t \rightsquigarrow (\text{One}\ \sigma', \text{false}, v)}\ \text{Stop}_\text{t}^\text{o}$$

$$\frac{\boxed{\checkmark}\ t \quad \text{next\_subst}\ \sigma\ t \rightsquigarrow \sigma' \quad \text{prune true}\ t \rightsquigarrow \text{Some}\ t'}{\text{run}_\text{t}\ p\ v\ \sigma\ t \rightsquigarrow (\text{Many}\ (\sigma', t'), \text{false}, v)}\ \text{Stop}_\text{t}^\text{m} \qquad \frac{\boxdot\ t \quad \text{step}\ p\ v\ \sigma\ t \rightsquigarrow (v', tg, t') \quad \text{run}_\text{t}\ p\ v'\ \sigma\ t' \rightsquigarrow (r, b, v'')}{\text{run}_\text{t}\ p\ v\ \sigma\ t\ \rightsquigarrow (r, (tg = \text{CutBrothers}) \vee b, v'')}\ \text{Step}_\text{t}$$

$$\frac{\boxtimes\ t \quad \text{prune false}\ t \rightsquigarrow \text{Some}\ t' \quad \text{run}_\text{t}\ p\ v\ \sigma\ t' \rightsquigarrow r}{\text{run}_\text{t}\ p\ v\ \sigma\ t \rightsquigarrow r}\ \text{Back}_\text{t} \qquad \frac{\text{prune false}\ t \rightsquigarrow \text{None}}{\text{run}_\text{t}\ p\ v\ \sigma\ t \rightsquigarrow (\text{None}, \text{false}, v)}\ \text{Fail}_\text{t}$$

Figure 6a. Rule system for run_t

$$\frac{A \quad (A\ \text{Cuts}_\text{s})}{A \vee B}\ (\vee_{il}) \qquad \frac{\neg A \quad (A\ \text{Cuts}_\text{s})}{\neg(A \vee B)}\ (\vee_{ir1}) \qquad \frac{\neg A \qquad B \quad (A\ \text{does not cut})}{A \vee B}\ (\vee_{ir2})$$

$$\frac{A \vee B \qquad \begin{matrix}[\text{A}] \\ C\end{matrix} \qquad (A\ \text{Cuts}_\text{s})}{C}\ (\vee_{e1}) \qquad \frac{A \vee B \qquad \begin{matrix}[\text{A}] \\ C\end{matrix} \qquad \begin{matrix}[\neg A, B] \\ C\end{matrix} \quad (A\ \text{does not cut})}{\text{C}}\ (\vee_{e2})$$

Figure 6b. Introduction and elimination rules for disjunction (Theorems 4.1 to 4.5)

the failed path; if the resulting cleaned tree exists, run_t is called recursively on it. Finally, rule Fail_t accounts for trees with no solutions.

4.4.1 Properties of run_t.

The boolean output of run_t allows us to state interesting properties about the Or node in the presence of cut (fig. 6b). To simplify their presentation, we introduce the following notation for Or nodes. When the left-hand side is None, we write the symbol □. Otherwise, the notation $A \vee B_\sigma$ implicitly denotes $(\text{Some}\ A) \vee B_\sigma$.

Theorem 4.1 (or_intro_left). $\forall\ p\ v\ v'\ \sigma\ \sigma'\ A\ A'\ \sigma_1\ B,$
$\text{run}_\text{t}\ p\ v\ \sigma\ A\ (\text{Many}\ \sigma'\ A')\ \text{true}\ v' \rightarrow$
$\text{run}_\text{t}\ p\ v\ \sigma\ (A \vee B_{\sigma_1})\ (\text{Many}\ \sigma'\ (A' \vee \text{Bot}_{\sigma_1}))\ \text{false}\ v'$

Theorem 4.2 (or_intro_right). $\forall\ p\ v\ v'\ \sigma\ A\ \sigma_1\ B,$
$\text{run}_\text{t}\ p\ v\ \sigma\ A\ \text{Zero true}\ v' \rightarrow \text{run}_\text{t}\ p\ v\ \sigma\ (A \vee B_{\sigma_1})\ \text{Zero false}\ v'$

Theorem 4.3 (or_intro_right2). $\forall\ p\ v_0\ v_1\ v_2\ \sigma\ A\ \sigma_1\ \sigma_2\ B\ b,$
$\text{run}_\text{t}\ p\ v_0\ \sigma\ A\ \text{Zero false}\ v_1 \wedge \text{run}_\text{t}\ p\ v_1\ \sigma_1\ B\ (\text{One}\ \sigma_2)\ b\ v_2 \rightarrow$
$\text{run}_\text{t}\ p\ v_0\ \sigma\ (A \vee B_{\sigma_1})\ (\text{One}\ \sigma_2)\ \text{false}\ v_2$

Theorems 4.1 to 4.3 correspond to the introduction rules for disjunction. If A executes a cut, one *cannot* obtain $A \vee B$ from B: the execution of A, whether it ultimately succeeds or fails, makes the success of B irrelevant (the cut discards it). In the first lemma, B is forced to be Bot; in the second one, the failure of A absorbs B. The last lemma is connected to the depth-first exploration of the tree: proving a disjunction from its right-hand side can only occur after the left-hand side has been disproved.

Theorems 4.4 and 4.5 are elimination rules for disjunction:

Theorem 4.4 (or_elim1). $\forall\ p\ v\ v'\ \sigma\ \sigma'\ \sigma_1\ A\ A'\ B\ B',$
$\text{run}_\text{t}\ p\ v\ \sigma\ (A \vee B_{\sigma_1})\ (\text{Many}\ \sigma'\ (A' \vee B'_{\sigma_1}))\ \text{false}\ v' \rightarrow$
$\exists\, b,\ \text{run}_\text{t}\ p\ v\ \sigma\ A\ (\text{Many}\ \sigma'\ A')\ b\ v' \wedge B' = \text{if}\ b\ \text{then}\ \bot\ \text{else}\ B$

Theorem 4.5 (or_elim2). $\forall\ p\ v\ v'\ \sigma\ \sigma'\ \sigma_1\ A\ B\ B',$
$\text{run}_\text{t}\ p\ v\ \sigma\ (A \vee B_{\sigma_1})\ (\text{Many}\ \sigma'\ (\square\ \vee B'_{\sigma_1}))\ \text{false}\ v' \rightarrow$
$(\text{run}_\text{t}\ p\ v\ \sigma\ A\ (\text{One}\ \sigma')\ \text{false}\ v') \vee$
$(\exists\ v_2\ b,\ \text{run}_\text{t}\ p\ v\ \sigma\ A\ \text{Zero false}\ v_2 \wedge$
$\text{run}_\text{t}\ p\ v_2\ \sigma_1\ B\ (\text{Many}\ \sigma'\ B')\ b\ v')$

From $A \vee B$, when A is not inert (i.e. not already explored), we cannot deduce A or B independently. Instead, we can only derive a weakened conclusion of the form $\top \vee B'$, where B' provides no information in the case where the exploration of A performs a cut. This yields an elimination rule that is stronger than the usual one. If A does not cut, the elimination rule has the usual two premises, together with the additional constraint that whenever B holds, A must have failed. This again reflects the depth-first traversal of the proof tree.

It is worth recalling that the "A cuts" side-condition in the rules above is precisely the boolean value returned by run_t. Without this information, it is impossible to formulate these rules: it is not merely a matter of whether A contains a syntactic occurrence of cut, but whether that cut is actually executed during the (dis)proof of A (in rules $(\vee_{ir1})$ and $(\vee_{ir2})$). Indeed, an atom preceding the cut may fail, so the cut is never reached.

As anticipated in the introduction, these lemmas show that, in the presence of cut, disjunction is not commutative.

4.5 Inadequacy of the stack-based semantics for formal reasoning

The lemmas that we presented in section 4.4.1 are hard to express in the stack-based semantics, due to the lack of structure in the stack to delimit the scope of the cut operator. For example, consider a stack $a_0, a_1, \ldots, a_n$. If a_0 succeeds but executes a cut, it is hard to relate the remaining alternatives $a_1, \ldots, a_n$ to the alternatives $b_1, \ldots, b_m$ that survive the cut, since the only simple invariant is that $b_1, \ldots, b_m$ is a suffix of $a_1, \ldots, a_n$. If $a_1, \ldots, a_n$ were generated before the call whose execution generates the cut in a_0, then they all stay; but if some were generated afterwards, they are discarded. Expressing this precisely would require attaching, at the very least, time-stamps (or depths) to goals, which amounts to a cumbersome encoding of a tree.

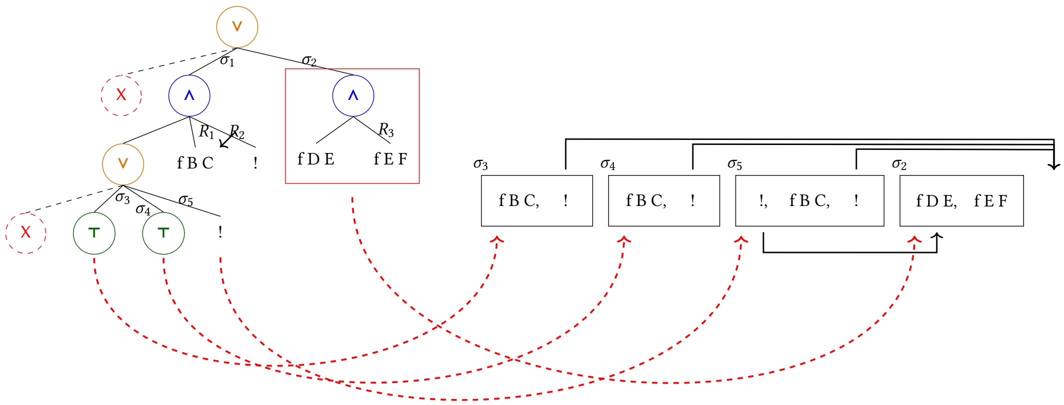


Figure 7. Example of t2s execution

5 Equivalence of the two semantics

Before going into the proof of the two semantics' equivalence, we need to relate trees to stacks.

5.1 From trees to stacks

The function t2s takes as input the tree t_0, a substitution σ for the input tree, and an auxiliary list of choice points cp.

$$\text{t2s} : \mathcal{T} \to \Sigma \to \text{alts} \to \text{alts}$$

These choice points represent alternatives that appear at the same level of the current tree, such as the right siblings of an Or node.

The Top node represents one successful alternative; therefore it is translated into a singleton list whose only element has an empty list of goals. The Bot node represents failure, hence it is mapped to the empty list of alternatives. Finally, atoms are mapped to a singleton list containing one alternative whose only goal is that atom, with an empty cut-to list. At this stage, the cut-to list cannot point to cp: atoms are not right-uncle subtrees, whereas cp represents alternatives that will actually be discarded if the atom turns out to be a cut. The correct cut-to list is therefore determined later, once the trees corresponding to sibling subtrees have been inspected.

The translation of $A \vee B_\sigma$ creates two lists of alternatives, one for A and the other for B. The alternatives of B are valid choice points for A and should be passed to the translation of A. The two lists of alternatives can then be concatenated. Moreover, every atom occurring one level below cp (i.e. in A or B) should add cp as its cut-to alternatives.

The translation of $A \wedge_{B_0} B$ starts with the translation of A. If the translation of A is an empty list, we return the empty list of alternatives without even touching B or B_0, since an empty translation for A indicates failure, which in turn implies the failure of the entire conjunction. When the translation of A is a list $(\sigma_0, g) :: a$, the B node represents the continuation of the first alternative g, whereas B_0 is the continuation for each alternative in a.

The function t2s is not injective: distinct trees can be translated into the same stack. In particular, a transition from a tree t_1 to a tree t_2 performed by run_t may result in both trees being translated into the same stack. We refer to such a transition as a *silent transition.*

Notably, the stack semantics has no notion of a failed stack. Therefore, when proving the equivalence between the two semantics, we must take into account that run_t may perform more iterations than run_s before returning a solution or reporting a failure. We illustrate this non-injective behavior with a concrete example below.

5.1.1 Example of t2s execution.

The translations of the trees in fig. 5 are shown in fig. 3. The letters in the figures relate each tree to the corresponding stack. For example, 5b is translated into 3b. We also note that 5d.1 and 5d.2 are both translated into 3d, showing that t2s is not injective: different trees can map to the same stack. Consequently, there are transitions in run_t that are not visible in run_s. As an example of a call to t2s, we take the tree and the stack in fig. 7, which are respectively taken from figs. 3c and 5c. The red arrows point from the head of an alternative in the tree to the head of the corresponding cell in the stack. We focus on the deepest Or node in the tree. This subtree is a disjunction with four children. The first is ignored, since it is None. The success node below σ_3 has `f B C` and the cut operator as continuation, corresponding to the first cell in the stack. We note that the cut operator points outside the stack, since the scope of this cut eliminates its right uncle. The success node below σ_4 has R_1 as continuation, where R_1 is the list of goals `f B C, !`; this is translated into the second cell of the stack. Finally, the cut operator under σ_5 also has R_1 as continuation. It corresponds to the third alternative. We can see that the first cut points to the translation of the subtree under σ_2, since it is one Or-layer above its right uncles.

5.2 Equivalence

The equivalence we are interested in states that a query produces the same substitution and alternatives under both semantics.

The tree-based semantics exposes more information than is needed for this comparison, so we hide the surplus behind existential quantifiers.

Definition 5.1 (run$'_t$). $\lambda\ p\ v\ \sigma\ t\ r.$
$\exists\ b\ v',\ \mathrm{run_t}\ p\ v\ \sigma\ (\mathrm{Unexplored}\ t)\ r\ b\ v'$

For each possible outcome, we prove the equivalence of the two semantics.

Corollary 5.2 (equiv_zero). $\forall\ p\ t\ (v := \mathrm{vars}\ t),$
$\mathrm{run_s}\ p\ v\ [\varepsilon,\ [\mathrm{call}\ t,\ []]]\ \mathrm{None} \leftrightarrow$
$\mathrm{run}'_t\ p\ v\ \varepsilon\ (\mathrm{call}\ t)\ \mathrm{Zero}$

Corollary 5.3 (equiv_one). $\forall\ p\ t\ \sigma'\ (v := \mathrm{vars}\ t),$
$\mathrm{run_s}\ p\ v\ [\varepsilon,\ [\mathrm{call}\ t,\ []]]\ (\mathrm{Some}\ (\sigma',\ [])) \leftrightarrow$
$\mathrm{run}'_t\ p\ v\ \varepsilon\ (\mathrm{call}\ t)\ (\mathrm{One}\ \sigma')$

Corollary 5.4 (sound_many). $\forall\ p\ t \sigma'\ t'\ (v := \mathrm{vars}\ t),$
$\mathrm{run}'_t\ p\ v\ \varepsilon\ (\mathrm{call}\ t)\ (\mathrm{Many}\ \sigma'\ t') \longrightarrow$
$\mathrm{run_s}\ p\ v\ [\varepsilon,\ [\mathrm{call}\ t,\ []]]\ (\mathrm{Some}\ (\sigma',\ \mathrm{t2s}\ t'\ \sigma\ []))$

Corollary 5.5 (complete_many). $\forall\ p\ t\ \sigma'\ x\ xs\ (v := \mathrm{vars}\ t),$
$\mathrm{run_s}\ p\ v\ [\varepsilon,\ [\mathrm{call}\ t,\ []]]\ (\mathrm{Some}\ (\sigma',\ x :: xs)) \longrightarrow$
$\exists\ t',\ \mathrm{run}'_t\ p\ v\ \varepsilon\ (\mathrm{call}\ t)\ (\mathrm{Many}\ \sigma'\ t') \wedge \mathrm{t2s}\ t'\ \varepsilon\ [] = x :: xs$

Because both semantics rely on the same notion of substitution, corollaries 5.2 and 5.3 can be stated without relating the states — the stacks and trees — on which the two semantics operate. In contrast, in corollaries 5.4 and 5.5 the execution leaves choice points open, so we do need to relate the two kinds of states.

Following the proof technique of [3], we only need the mapping t2s from trees to stacks, already described in section 5.1; the converse mapping is never constructed explicitly. The corollaries above follow from the two theorems proved in section 5.3.

5.3 Equivalence proof

The two theorems below each proceed by induction on the execution of one semantics — the top or the bottom horizontal arrow in the diagram — in order to derive the other. We begin with the top-to-bottom direction, from trees to stacks.

$$\begin{tikzcd} t \arrow[r, "\mathrm{run_t}'\ t"] \arrow[d, "\mathrm{t2s}"'] & t' \arrow[d, "\mathrm{t2s}"] \\ a \arrow[r, "\mathrm{run_s}\ a"'] \arrow[ur] & a' \end{tikzcd}$$

Since not every tree or stack can be produced by either semantics starting from the initial state, it is important to identify the subset of valid trees. We defer the exact definition of SLD_tree to section 5.3.1.

Theorem 5.6 (runT2S). $\forall\ p\ t\ \sigma\ r\ (v := \mathrm{vars}\ t\ \cup\ \mathrm{vars}\ \sigma),$
$\mathrm{sld\_tree}\ t \wedge (\exists\ b\ v',\ \mathrm{run_t}\ p\ v\ \sigma\ t\ r\ b\ v') \longrightarrow$

$$\begin{cases} \mathrm{run_s}\ p\ v\ (\mathrm{t2s}\ t\ \sigma\ [])\ \mathrm{None} & \text{if } r = \mathrm{Zero} \\ \mathrm{run_s}\ p\ v\ (\mathrm{t2s}\ t\ \sigma\ [])\ (\mathrm{Some}\ (\sigma', [])) & \text{if } r = \mathrm{One}\ \sigma' \\ \mathrm{run_s}\ p\ v\ (\mathrm{t2s}\ t\ \sigma\ [])\ (\mathrm{Some}\ (\sigma',\ \mathrm{t2s}\ t'\ \sigma\ [])) & \text{if } r = \mathrm{Many}\ \sigma'\ t' \end{cases}$$

Proof. Induction on $\mathrm{run_t}$. □

The converse direction could in principle be proved symmetrically, by defining a mapping from stacks to trees together with a predicate identifying the valid stacks that the semantics can generate from the initial one. The backward simulation technique of [3] that we follow makes both definitions unnecessary, by using an existential quantifier instead (on t' in the third case). Validity of the stack a is recovered for free, since it is obtained from a valid SLD tree; and by inhabiting the existential in a constructive logic, we implicitly obtain the mapping from stacks to trees.

Theorem 5.7 (runS2T). $\forall\ p\ v\ a\ r,$
$\mathrm{run_s}\ p\ v\ a\ r \longrightarrow \forall\ \sigma\ t,\ \mathrm{sld\_tree}\ t \longrightarrow \mathrm{t2s}\ t\ \sigma\ [] = a \longrightarrow \exists\ b\ v',$

$$\begin{cases} \mathrm{run_t}\ p\ v\ \sigma\ t\ \mathrm{Zero}\ b\ v' & \text{if } r = \mathrm{None} \\ \mathrm{run_t}\ p\ v\ \sigma\ t\ (\mathrm{One}\ \sigma')\ b\ v' & \text{if } r = \mathrm{Some}\ (\sigma',\ []) \\ \exists\ t',\ \mathrm{run_t}\ p\ v\ \sigma\ t\ (\mathrm{Many}\ \sigma'\ t')\ b\ v' & \text{if } r = \mathrm{Some}\ (\sigma',\ a') \\ \qquad \wedge\ \mathrm{t2s}\ t'\ \sigma\ [] = a' & \end{cases}$$

Proof. Induction on $\mathrm{run_s}$. Moreover, in order to prove this theorem, we need an auxiliary lemma to align the iterations in $\mathrm{run_s}$ with the corresponding iterations in $\mathrm{run_t}$ by accounting for silent transitions. □

5.3.1 Valid SLD trees.

The class of valid SLD trees is captured by the SLD_tree function shown in fig. 8. While all base cases of tree are valid, we need to analyze the Or and And constructors more carefully.

For the Or constructor, we distinguish two cases depending on whether the left-hand side is present. If it is not, then the right-hand side must be a valid SLD tree. Otherwise, the left-hand side must be a valid SLD tree, and the right-hand side must either be the Bot tree or be unexplored. The former case occurs when the right-hand side is cut away by the evaluation of a superficial cut in the left-hand side. In the

```
Fixpoint sld_tree A :=
  match A with
  | Unexplored _ | Top | Bot ⇒ true
  | Or None _ B ⇒ sld_tree B
  | Or (Some A) _ B ⇒
      sld_tree A && ((B == Bot) || base_or B)
  | And A B0 B ⇒
      sld_tree A && if success A then sld_tree B
                                  else B == big_and B0
  end.
```

Figure 8. Valid SLD tree

latter case we use the *base_or* predicate to check that B is the right-skewed tree formed by a disjunction of conjunctions, generated after a successful backchain for a call.

For the And constructor, the left-hand side is required to be a valid SLD tree. The shape of the right-hand side depends on whether the left-hand side is a success. If it is not successful, then the right-hand side must not be explored, i.e. B must be the right-skewed tree containing conjunctions of premise atoms. This is enforced using the *big_and* procedure, which generates a tree from the reset point B_0 (the same procedure used to transform each alternative output by backchain into the And-layer of the resulting tree). When the left-hand side is successful, the right-hand side may have been explored, and consequently must be a valid SLD tree.

We point out the two main invariants stating that valid SLD trees are preserved by calls to step and prune.

Lemma 5.8 (sld_tree_step). $\forall\ \sigma\ v\ A\ r,$
$\text{sld\_tree}\ A \to \text{step}\ u\ p\ v\ \sigma\ A = r \to \text{sld\_tree}\ (\text{snd}\ r)$

Lemma 5.9 (sld_tree_prune). $\forall\ A\ B\ b,$
$\text{sld\_tree}\ A \to \text{prune}\ b\ A = \text{Some}\ B \to \text{sld\_tree}\ B$

Theorem 5.10 (sld_tree_run). $\forall\ b\ \sigma\ \sigma'\ v\ v'\ A\ B,$
$\text{sld\_tree}\ A \to \text{run}_t\ u\ p\ v\ \sigma\ A\ (\text{Many}\ \sigma'\ B)\ b\ v' \to \text{sld\_tree}\ B$

6 Case study: determinacy analysis

As an application of the tree semantics, we mechanize the soundness proof of the determinacy-checking algorithm introduced in version 3 of the Elpi language [1]. The static checker takes as input the signatures of the predicates declared by the user and verifies that the rules of a deterministic predicate generate at most one solution; we call this kind of predicate a function. Thanks to this static verification, the user can better control unexpected backtracking.

A predicate behaves deterministically if two conditions are satisfied: *mutual exclusion* guarantees that at most one rule can be successfully applied to any given query, whereas *rule-body checking* ensures that each rule does not create choice points, for example by calling non-deterministic predicates.

Our mechanization implements the unification algorithm by Martelli and Montanari [20]; we prove it terminating (using the standard lexicographic ordering), correct, and complete.

In the following definitions and lemmas, we use the notation $x \cap y = \varnothing$ to denote that the two sets x and y are disjoint. When the arguments are terms, as in $t_1 \cap t_2 = \varnothing$, we mean that the two terms do not share any variables.

The first notion to provide is the application of an acyclic substitution:

Definition 6.1 (acyclic). $\lambda\sigma.\text{dom}\ \sigma \cap \text{codom}\ \sigma = \varnothing$

Note that this definition of acyclicity entails that the substitution is necessarily self applied, i.e. $\sigma := \{X \mapsto f\ Y, Y \mapsto a\}$ is not acyclic, whereas the equivalent $\sigma' := \{X \mapsto f\ a, Y \mapsto a\}$ is. The algorithm by Martelli and Montanari generates substitutions that are acyclic in this sense.

Lemma 6.2 (unify_correct). $\forall\ t_1\ t_2\ \sigma\ \sigma',$
$\text{acyclic}\ \sigma \to \text{unify}\ t_1\ t_2\ \sigma = \text{Some}\ \sigma' \to \sigma'\ t_1 = \sigma'\ t_2$

Lemma 6.3 (unify_complete). $\forall\ t_1\ t_2\ \sigma,\ \text{acyclic}\ \sigma \to$
$(\exists\ \sigma',\ \text{acyclic}\ \sigma' \land\ \sigma'\ (\sigma\ t_1) = \sigma'\ (\sigma\ t_2)) \to$
$\exists\ \sigma'',\ \text{unify}\ t_1\ t_2\ \sigma = \text{Some}\ \sigma''$

The Elpi language adopts a restricted unification routine for input arguments, called matching [2, 31]. Like unify, matching takes a query term t_1 and a pattern t_2, but it also takes a set of frozen variables v and returns an optional substitution σ' that does not assign any variable in v.

To address this need, we extend the Martelli-Montanari unification procedure so that it takes a set of frozen variables. The function matching has the following property:

Lemma 6.4 (matching_disj). $\forall\ v\ t_1\ t_2\ \sigma\ \sigma', \to \text{vars}\ (\sigma\ t_2) \subseteq v$
$\text{acyclic}\ \sigma \to \text{matching}\ v\ t_1\ t_2\ \sigma = \text{Some}\ \sigma' \to \sigma'\ t_1 = \sigma\ t_2$

The backchain function uses matching or unify on the arguments q depending on whether the argument is in *input* or *output* mode.

For the static checker, and in particular in order to validate rule mutual exclusion, we need to ensure that no two rules can be used on the same query in the absence of cut. We exploit the behavior of the matching over input arguments to guarantee this property: mutual exclusion ensures that every pair of rules for a deterministic predicate has an input term that does not unify.

An important property we derive from the matching and unification procedures is the following. Given three pairwise variable-disjoint terms h_1, h_2, and q, representing two rule heads in the database and a query, if both heads match the query q, then the two heads must unify.

Lemma 6.5 (matching_unify_trans). $\forall\ v\ h_1\ h_2\ q,$
$h_1 \cap h_2 = \varnothing \land h_1 \cap q = \varnothing \land h_2 \cap q = \varnothing \land \text{vars}\ q \subseteq v \to$
$\text{matching}\ v\ h_1\ q\ \varnothing \land \text{matching}\ v\ h_2\ q\ \varnothing \to \text{unify}\ h_1\ h_2\ \varnothing$

Proof. By lemma 6.4, we know that the matching of h_1 (resp. h_2) and q gives a substitution σ_1 (resp. σ_2) where the variables in q are not assigned, i.e. its domain only contains variables in h_1 (resp h_2). Using lemma 6.3 we need to prove that it exists a substitution σ' such that $\sigma'\ h_1 = \sigma'\ h_2$. This substitution is the composition of σ_1 and σ_2, that is $\sigma' = \lambda x.\sigma_1(\sigma_2\ x)$. Since h_1 and h_2 are disjoint, so are the domains of σ_1 and σ_2, therefore $\sigma' h_1 = \sigma_2(\sigma_1\ h_1)$ and $\sigma' h_2 = \sigma_2(\sigma_1\ h_2)$. We can now conclude the proof: $\sigma'\ h_1 = \sigma_2(\sigma_1\ h_1) = \sigma_2\ q = q = \sigma_2\ h_2 = \sigma' h_2$. □

Finally, we also need to prove that the refreshing of variables performed at runtime does not affect unification. In particular, a refreshing renaming f for a term t is a function from variables to variables such that: (i) f is defined on all variables in t, i.e. every variable is renamed; and (ii) f is injective, i.e. it does not identify distinct variables. Refreshing therefore produces α-equivalent terms.

Definition 6.6 ($=_\alpha$)**.** $\lambda t_1\ t_2.$
$\exists (r : \text{renaming\_for } t_2), t_1 = \text{rename } r\ t_2$

Lemma 6.7 (unif_ren_ac)**.** $\forall\ t_1\ t_2\ t_1'\ t_2',$
$t_1 =_\alpha t_1' \land\ t_2 =_\alpha t_2' \land t_1 \cap t_2 = \varnothing \land\ t_1' \cap t_2' = \varnothing \rightarrow$
$\text{unify } t_1\ t_2\ \varepsilon \rightarrow \text{unify } t_1'\ t_2'\ \varepsilon$

In fig. 1, the rules of the program are equipped with three signatures, one per predicate. Besides the arity of each predicate, a signature specifies which arguments are inputs and which are outputs, their types, and whether the predicate is deterministic. They indicate that `f` and `g` are expected to be functions, as specified by the **`func`** keyword, whereas `r` is a relation, as indicated by the **`pred`** keyword. The **`->`** symbol separates inputs from outputs; therefore, all three predicates are expected to take an `int` as input and return an `int` as output.

The signatures of the program are stored in the `sig` field of the program and are encoded as described in [1]: we distinguish data from predicates, and predicates are classified as deterministic or non-deterministic.

A program execution behaves deterministically if, given a program, a substitution, and an input tree for the run_t inductive, the execution either fails or succeeds with no alternatives left to explore.

Definition 6.8 (is_det)**.** $\lambda\ p\ \sigma\ t,\ \forall\ r\ (v := \text{vars } t \cup \text{vars } \sigma),$
$\text{run}_s\ p\ v\ [\sigma,\ [t, []]]\ r \rightarrow r = \text{None} \lor \exists\ \sigma,\ r = \text{Some}\ (\sigma,\ [])$

Definition 6.9 (tm_is_det)**.** Given a program p and a term t, we say that t is deterministic if its head is a predicate f which is declared as deterministic in p.

The theorem we want to prove is the following one:

Theorem 6.10 (det_check_call)**.** $\forall\ p\ \sigma\ t,$
$\text{det\_checked } p \rightarrow \text{tm\_is\_det } p\ t \rightarrow \text{is\_det } p\ \sigma\ (\text{call } t)$

Here, det_checked denotes the function that checks the program with respect to mutual exclusion and rule-body checking.

Due to the inadequacy of the stack semantics wrt the scope of the cut, proving the previous lemma as it is stated is not an easy task. We need therefore to prove a lemma talking about determinacy declined on trees, and then reuse the theorems from section 5.2 to derive a proof of determinacy for stacks.

The lemma we prove is the following:

Lemma 6.11 (det_check_tree)**.** $\forall\ \sigma\ v\ p\ t\ r\ b\ v',$
$\text{det\_checked } p \rightarrow \text{det\_tree } p\ t \rightarrow$
$\text{run}_t\ p\ v\ \sigma\ t\ r\ b\ v' \rightarrow r = \text{Zero} \lor \exists\ \sigma,\ r = (\text{One}\ \sigma)$

The notion of deterministic trees, written det_tree, intuitively classifies trees whose interpretation does not generate choice points. The definition is shown in fig. 9.

```
Fixpoint det_tree (p: program) A :=
  match A with
  | Unexplored cut => true
  | Unexplored (call t) => tm_is_det p t
  | Bot | Top => true
  | And A B0 B =>
      det_tree p B &&
      ((prune (success A) A == None) ||
       ((det_tree p A || (has_cut B && has_cuts B0)) &&
        det_trees p B0))
  | Or None _ B => det_tree p B
  | Or (Some A) _ B =>
      det_tree p A &&
      if has_cut A then det_tree p B else (B == Bot)
  end.
```

Figure 9. The *det_tree* predicate

For a call to a term t, the tree is deterministic whenever t calls a deterministic predicate. The recursive cases correspond to the And and Or nodes.

A conjunction behaves deterministically whenever its right-hand side is a deterministic tree. Two cases must then be considered.

If prune (success A) A = None, then A produces no choice points: it either always fails or contains a single successful execution path. Since det_tree p B holds, the entire conjunction is therefore deterministic.

Otherwise, the left-hand side may introduce backtracking, potentially resuming the reset point, which must itself behave deterministically. In this case, either A is deterministic, or both B and the reset point contain a cut that prevents the non-deterministic behaviour arising from the evaluation of A.

For Or trees, the interesting case is when the left-hand side is present. In this case, A must be deterministic. If A contains a superficial cut, then B must also be deterministic. Indeed, if A succeeds, the cut discards B; otherwise, if A fails before reaching the cut, then no solution is produced by A and B is evaluated.

Finally, if A does not contain a cut, then B must be equal to $\bot$, that is, a tree already discarded by a cut. This condition is essential. Otherwise, if A succeeds without a superficial cut and B is not $\bot$, then B could still produce a successful branch, leading to non-deterministic behaviour.

The following auxiliary lemmas show that deterministic trees are preserved by the operations step and prune, and are used to prove lemma 6.11.

Lemma 6.12 (det_tree_step)**.** $\forall\ \text{p}\ v\ \sigma\ t\ r,$
$\text{det\_checked p} \rightarrow \text{step } u\ \text{p}\ v\ \sigma\ t = r \rightarrow \text{det\_tree p}\ (\text{snd } r)$

Lemma 6.13 (det_tree_prune)**.** $\forall\ p\ t\ t'\ b,$
$\text{det\_tree } p\ t \rightarrow \text{prune } b\ t = \text{Some } t' \rightarrow \text{det\_tree } p\ t'$

Finally, the proof for theorem 6.10 is as follows: from lemma 6.11, conclude that a call to a deterministic predicate behaves deterministically on trees, then, thanks to corollary 5.4, we derive the proof that a call to a detemrinistic predicate behaves deterministically when run in run_s.

7 Related work

The only mechanization of PROLOG semantics we are aware of is that of Pusch in ISABELLE/HOL [24], which, like ours, follows Debray and Mishra [8, Sec. 4.3]. Pusch starts from this semantics and refines it by introducing optimizations commonly found in the Warren abstract machine [2, 30]. Unlike our case study, she does not use the semantics to prove properties of program execution, and therefore does not encounter the difficulty described in section 3 that led us to develop a semantics in which the effect of cut is more explicit. In this sense, the two developments are complementary: taken together, they suggest that our tree-based semantics is equivalent to an optimized stack-based one.

Another relevant work is due to Kriener and King [17], although we could not find their ROCQ source code. Their semantics is denotational, which would have sufficed for the first-order fragment considered in this paper but cannot easily accommodate all of the features covered in [1]. More importantly, their mechanization does not relate the denotational semantics to an operational semantics of the kind underlying standard PROLOG abstract machines, a link that our equivalence proof provides.

Hanus and Prott [14] formalize in ROCQ a determinacy checker for the logic-functional language Curry. In this language, choice points are explicitly denoted by the `?` node in the syntax of expressions, so mutual exclusion of rules need not be established through unification: the overlap between rules is already explicit in the syntax. Similarly, the operator that absorbs choice points, `allValues`, explicitly takes as an argument the expression whose choice points it discards, which makes the scope of this control operator trivial to determine. Cut, by contrast, has no such syntactic marker: its scope is implicit in the surrounding stack or tree structure, which is precisely why relating the two semantics is nontrivial in our setting.

The proof technique we use to relate the two semantics is inspired by Bertot's mechanization of compiler correctness [3]. He relates the semantics of an imperative language to that of its compiled assembly code and proves their equivalence. His approach uses the compiler as the translation function from one language to the other but does not introduce a decompiler, just as we do not define a function from stacks to trees. To show that the assembly code behaves like the imperative language, he proceeds by induction on the execution of the assembly program; we adopt the same strategy in the proof of theorem 5.7.

For the static analysis of determinacy, we use a semantics in which input arguments are *matched*, rather than unified, against rule heads. This choice agrees with the ELPI interpreter, which is in turn inspired by B-PROLOG's "matching clauses" [2, 31]. Our results can nonetheless be applied to a semantics based entirely on unification, by adding two assumptions to theorem 6.10: the initial query must have ground inputs, and the program must be well-moded. Since well-moded predicates produce ground outputs from ground inputs, a well-moded program run from a query with ground inputs behaves exactly as if its input arguments were matched. In other words, lemma 6.4 could be reformulated in terms of unification at the cost of this ground-input assumption, under which unification and matching coincide.

8 Conclusion

This paper presents two mechanized operational semantics for PROLOG with cut, together with a proof of their equivalence. The first semantics, due to Debray and Mishra, is based on a stack of alternatives: it is close to actual PROLOG implementations, including the first-order fragment of ELPI, but it leaves the scope of cut implicit in the shape of the stack, which makes it inconvenient for reasoning about cut. The second semantics is novel and is based on And–Or trees, in which the scope of cut is explicit in the tree structure; it is well suited both to graphical illustrations of the effect of cut and to stating introduction and elimination lemmas for disjunction in its presence. We show how to translate trees into stacks and use this translation to prove the two semantics equivalent. Building on the tree-based semantics, we mechanize a soundness proof for the first-order fragment of the determinacy checker of [1]: a call to a predicate identified as deterministic returns at most one solution.

The mechanization comprises approximately 2,500 lines of specifications and 5,000 lines of SSREFLECT proofs, built on the Mathematical Components library [19] and its finmap [21] extension. Roughly 30% of the proof code is devoted to the tree semantics, in particular the proofs in section 4.4; 15% to the stack semantics; 35% to the equivalence proof; and the remaining 20% to the determinacy checker. The mechanization took us approximately 8 months. We believe that the SSREFLECT proof language, and the proof style it encourages, made it easier to carry out the many refactorings the development underwent. The finmap library, in particular, provides comprehensive support for reasoning about finite maps and finite sets over a countable domain; it helped us express conditions such as the disjointness between a set of variables and the domain of a substitution, and to define the union of disjoint substitutions.

To keep the mechanization axiom-free, we implemented the first-order unification algorithm ourselves, since we could not find an existing ROCQ mechanization; a detailed account of this development is beyond the scope of this paper. It amounts to about 1,700 lines of SSREFLECT code, including 200 lines for a termination proof based on a lexicographic ordering and 800 lines for correctness and completeness. Because lemmas 6.4 and 6.7 only involve matching and unification of terms with disjoint support, we did not need to prove that the algorithm produces most general unifiers.

Fully modeling the Elpi language, and thereby mechanizing [1] in its entirety, requires accounting for higher-order variables, i.e., variables that stand for predicates. This extension is ongoing work, largely orthogonal to the present paper. Its main difficulty is that substitutions may then bind variables to predicates, whose signatures must remain compatible with those assumed statically, in the sense of the $\subseteq$ relation defined in [1, Section 5]; accommodating dynamic predicates also requires minor changes to the mutual exclusion conditions.

References

[1] Anonymized. 2026. Determinacy Checking for Elpi: an Higher-Order Logic Programming Language with Cut. In *Practical Aspects of Declarative Languages*, Nada Amin and Joaquín Arias (Eds.). Springer Nature Switzerland, Cham, 77–95.

[2] Hassan Aït-Kaci. 1991. *Warren's Abstract Machine: A Tutorial Reconstruction*. The MIT Press. doi:10.7551/mitpress/7160.001.0001

[3] Yves Bertot. 1998. *A certified Compiler for an Imperative Language*. Technical Report RR-3488. INRIA. 30 pages. https://inria.hal.science/inria-00073199v1

[4] Valentin Blot, Denis Cousineau, Enzo Crance, Louise Dubois de Prisque, Chantal Keller, Assia Mahboubi, and Pierre Vial. 2023. Compositional Pre-processing for Automated Reasoning in Dependent Type Theory. In *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2023, Boston, MA, USA, January 16-17, 2023*, Robbert Krebbers, Dmitriy Traytel, Brigitte Pientka, and Steve Zdancewic (Eds.). ACM, 63–77. doi:10.1145/3573105.3575676

[5] Cyril Cohen, Enzo Crance, and Assia Mahboubi. 2024. Trocq: Proof Transfer for Free, With or Without Univalence. In *Programming Languages and Systems - 33rd European Symposium on Programming, ESOP 2024, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2024, Luxembourg City, Luxembourg, April 6-11, 2024, Proceedings, Part I (Lecture Notes in Computer Science)*, Stephanie Weirich (Ed.). Springer, 239–268. doi:10.1007/978-3-031-57262-3_10

[6] Cyril Cohen, Enzo Crance, and Assia Mahboubi. 2025. Trocq: Proof Transfer for Free, Beyond Equivalence and Univalence. *ACM Trans. Program. Lang. Syst.* 47, 3 (2025), 12:1–12:40. doi:10.1145/3737283

[7] Cyril Cohen, Kazuhiko Sakaguchi, and Enrico Tassi. 2020. Hierarchy Builder: Algebraic hierarchies Made Easy in Coq with Elpi. In *Proceedings of FSCD (LIPIcs, Vol. 167)*. 34:1–34:21. doi:10.4230/LIPIcs.FSCD.2020.34

[8] Saumya K. Debray and Prateek Mishra. 1988. Denotational and operational semantics for Prolog. *J. Log. Program.* 5, 1 (March 1988), 61–91. doi:Debray1988

[9] Saumya K. Debray and David S. Warren. 1989. Functional computations in logic programs. *ACM Trans. Program. Lang. Syst.* 11, 3, 451–481. doi:10.1145/65979.65984

[10] Cvetan Dunchev, Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. 2015. ELPI: Fast, Embeddable, λProlog Interpreter. In *Proceedings of LPAR (LNCS, Vol. 9450)*. Springer, 460–468. doi:10.1007/978-3-662-48899-7_32

[11] Davide Fissore and Enrico Tassi. 2023. A new Type-Class solver for Coq in Elpi. In *The Coq Workshop*. https://inria.hal.science/hal-04467855

[12] Davide Fissore and Enrico Tassi. 2024. Higher-Order unification for free!: Reusing the meta-language unification for the object language. In *Proceedings of PPDP*. ACM, Article 3, 13 pages. doi:10.1145/3678232.3678233

[13] Benjamin Grégoire, Jean-Christophe Léchenet, and Enrico Tassi. 2023. Practical and Sound Equality Tests, Automatically. In *Proceedings of CPP*. Association for Computing Machinery, 167–181. doi:10.1145/3573105.3575683

[14] Michael Hanus and Kai-Oliver Prott. 2025. Determinism Types for Functional Logic Programming. In *Proceedings of PPDP*. 47–59.

[15] Fergus Henderson, Zoltan Somogyi, and Thomas Conway. 1996. Determinism analysis in the Mercury compiler. In *Proceedings of Australian Computer Science Conference*. 337–346.

[16] Robbert Krebbers, Luko van der Maas, and Enrico Tassi. 2025. Inductive Predicates via Least Fixpoints in Higher-Order Separation Logic. In *16th International Conference on Interactive Theorem Proving (ITP 2025) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 352)*, Yannick Forster and Chantal Keller (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 27:1–27:21. doi:10.4230/LIPIcs.ITP.2025.27

[17] Jael Kriener and Andy King. 2011. RedAlert: Determinacy inference for Prolog. *Theory and Practice of Logic Programming* 11, 4-5, 537–553.

[18] P. López-García, F. Bueno, and M. Hermenegildo. 2005. Determinacy Analysis for Logic Programs Using Mode and Type Information. In *Logic Based Program Synthesis and Transformation*, Sandro Etalle (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 19–35.

[19] Assia Mahboubi and Enrico Tassi. 2022. *Mathematical Components*. Zenodo. doi:10.5281/zenodo.7118596

[20] Alberto Martelli and Ugo Montanari. 1982. An Efficient Unification Algorithm. *ACM Trans. Program. Lang. Syst.* 4, 2 (April 1982), 258–282. doi:10.1145/357162.357169

[21] math-comp. 2026. finmap — Finite maps for MathComp. https://github.com/math-comp/finmap

[22] Dale Miller. 1991. A logic programming language with lambda-abstraction, function variables, and simple unification. In *Extensions of Logic Programming*. Springer, 253–281.

[23] Dale Miller and Gopalan Nadathur. 2012. *Programming with Higher-Order Logic*. Cambridge University Press.

[24] Cornelia Pusch. 1996. Verification of compiler correctness for the WAM. In *Theorem Proving in Higher Order Logics*, Gerhard Goos, Juris Hartmanis, Jan van Leeuwen, Joakim von Wright, Jim Grundy, and John Harrison (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 347–361.

[25] Zoltan Somogyi, Fergus Henderson, and Thomas Conway. 1996. The execution algorithm of mercury, an efficient purely declarative logic programming language. *The Journal of Logic Programming* 29, 1, 17–64. doi:10.1016/S0743-1066(96)00068-4 High-Performance Implementations of Logic Programming Systems.

[26] SWI-Prolog Documentation. 2026. Predicate `!/0` — Cut (built-in predicate). https://www.swi-prolog.org/pldoc/doc_for?object=!/0 Accessed via SWI-Prolog PlDoc reference.

[27] Enrico Tassi. 2018. Elpi: an extension language for Coq (Metaprogramming Coq in the Elpi λProlog dialect). In *The Fourth International Workshop on Coq for Programming Languages*. https://inria.hal.science/hal-01637063

[28] Enrico Tassi. 2019. Deriving proved equality tests in Coq-Elpi. In *Proceedings of ITP (LIPIcs, Vol. 141)*. 29:1–29:18. doi:10.4230/LIPIcs.CVIT.2016.23

[29] Luko van der Maas. 2024. *Extending the Iris Proof Mode with Inductive Predicates using Elpi*. Master's thesis. Radboud University Nijmegen. doi:10.5281/zenodo.12568604

[30] David H.D. Warren. 1983. *An Abstract Prolog Instruction Set*. Technical Report Technical Note 309. SRI International, Artificial Intelligence Center, Computer Science and Technology Division, Menlo Park, CA, USA. https://www.sri.com/wp-content/uploads/2021/12/641.pdf

[31] Neng-fa Zhou. 2012. The language features and architecture of b-prolog. *Theory Pract. Log. Program.* 12, 1–2 (Jan. 2012), 189–218. doi:10.1017/S1471068411000445

A The step procedure (file tree.v, line 200)

```
Fixpoint step p v s A :=
  match A with
  | Top                 => (v, Success, Top)
  | Bot                 => (v, Failed, A)
  | Unexplored cut      => (v, CutBrothers, Bot)
  | Unexplored (call t) =>
      let: (v, l) := backchain u p v t s in
      (v, Expanded,
      if l is ((s, r) :: xs) then Or None s (big_or r xs) else Bot)
  | Or A sB B =>
      if A is Some A then
        let: (v, tA, rA) := step p v s A in
        (v, if is_cb tA then Expanded else tA,
          Or (Some rA) sB (if is_cb tA then Bot else B))
      else
        let: (v, tB, rB) := step p v sB B in
        (v, if is_cb tB then Expanded else tB, Or A sB rB)
  | And A B0 B =>
      if success A then
        let: (v, tB, rB) := step p v (next_subst s A) B in
        (v, tB, And (if is_cb tB then cutl A else A) B0 rB)
      else
        let: (v, tA, rA) := step p v s A in (v, tA, And rA B0 B)
  end.
```

B The prune procedure (file tree.v, line 243)

```
Definition omap f o := match o with Some b => Some (f b) | _ => o end.
Fixpoint prune b A : option tree :=
  match A with
  | Bot => None
  | Top => if b then None else Some Top
  | Unexplored _ => Some A
  | And A B0 B =>
      let build_B0 b := omap (λx. And x B0 (big_and B0)) (prune b A) in
      if success A then
        if prune b B is Some B' then Some (And A B0 B')
        else build_B0 true
      else if failed A then build_B0 false
      else Some (And A B0 B)
  | Or None sB B => omap (λx. Or None sB x) (prune b B)
  | Or (Some A) sB B =>
      if prune b A is Some A' then Some (Or (Some A') sB B)
      else omap (λx. Or None sB x) (prune false B)
end.
```